Sentinel: Real-Time Kafka Pipeline

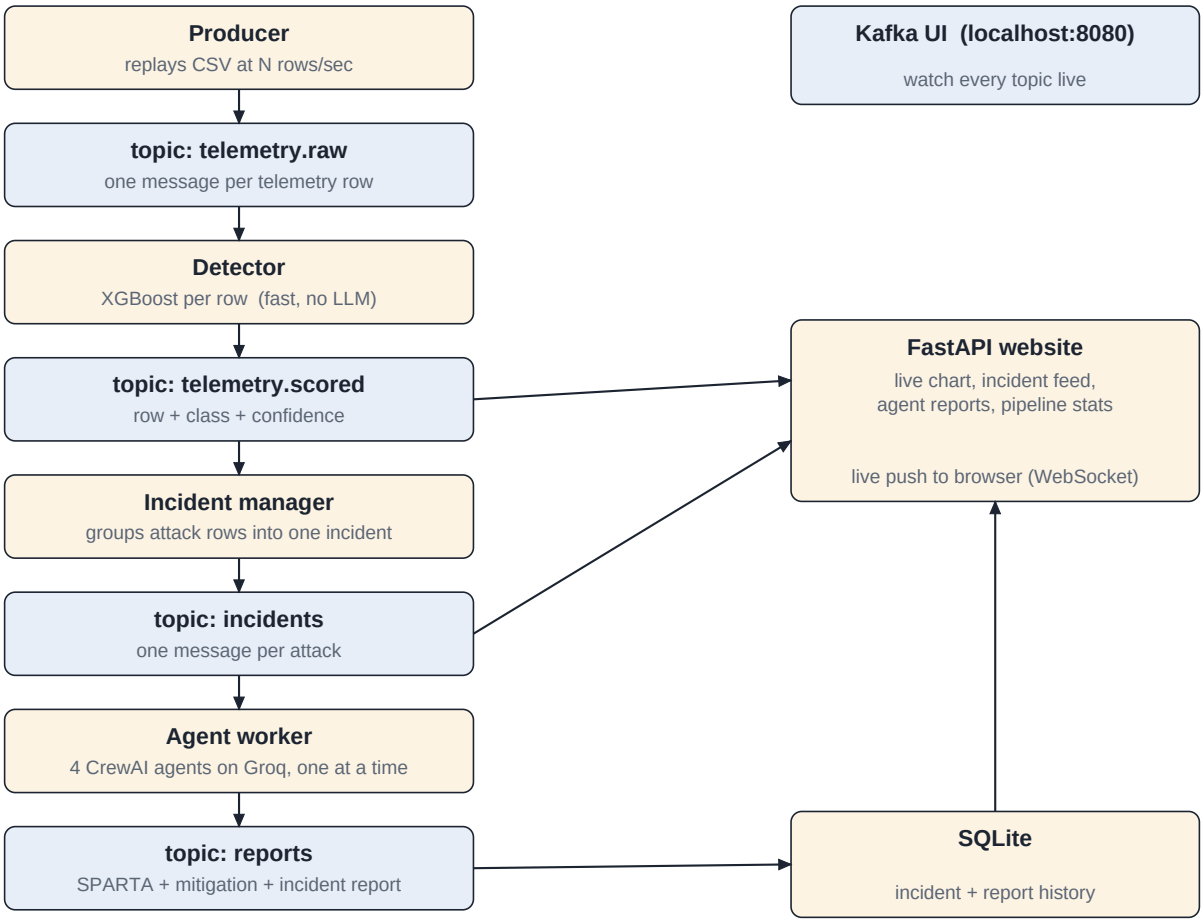
Implementation plan, to be completed in one day (6 Nov 2026) · IIT Bombay poster build

1. Goal

Turn Sentinel from a click-to-analyse dashboard into a **real-time monitoring system**. Telemetry streams through Apache Kafka, every row is classified the moment it arrives, attack bursts become single incidents, the 4-agent AI crew analyses each incident automatically, and a new website shows all of it live. The existing Gradio dashboard keeps working while this is built; the Hugging Face Space is removed at the end.

2. Architecture

Each box on the left is a small, independent Python service. Services never call each other: they read from one Kafka topic and write to the next. If the slow AI agents fall behind, incidents wait safely in their topic while detection keeps running at full speed.



Blue = Kafka topic, beige = service.

3. Components

Component	What it does	Built from
Kafka (KRaft) + Kafka UI	Message broker in Docker, single node, no Zookeeper. Kafka UI shows topics and messages live.	docker-compose.yml (apache/kafka, kafbat/kafka-ui)
Producer	Replays the dataset CSV into telemetry.raw at a set speed; can inject attack bursts for demos.	Existing TelemetryReplaySimulator
Detector	Consumes telemetry.raw, runs XGBoost on every row, publishes class + confidence to telemetry.scored.	Existing classify_telemetry()

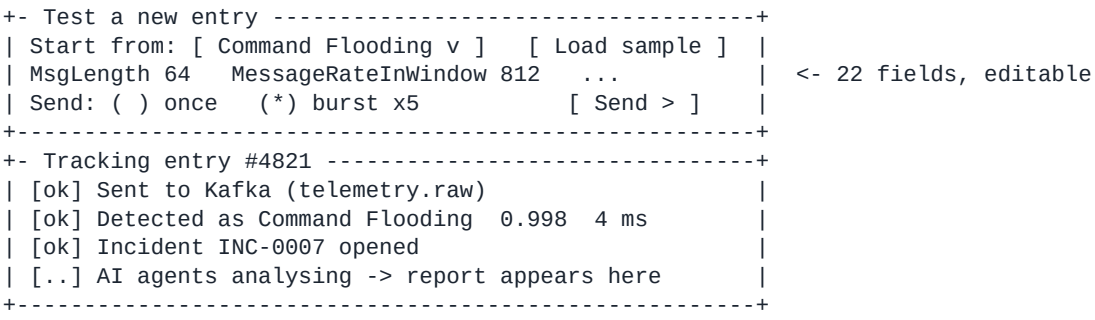
Component	What it does	Built from
Incident manager	Uses the class XGBoost predicted; it never re-classifies. Consecutive non-Normal rows of the same class within a time gap become one incident; publishes when the incident opens and when it closes. **False-alarm guards** (configurable): a row counts only if confidence ≥ 0.8, and an incident opens only after 3 consecutive attack rows of the same class.	New logic
Agent worker	Consumes incidents one at a time, runs the 4-agent crew (Classifier, SPARTA, Mitigation, Reporter), publishes to reports. Respects the Groq rate limit.	Existing full_pipeline.py + RetryingLLM
Storage	Saves incidents and reports to SQLite so the website can show history after a restart.	New, Python sqlite3
Website	FastAPI backend consuming Kafka; browser updated over WebSocket: live telemetry chart, incident feed, agent report viewer, throughput/lag stats.	New: FastAPI + one HTML/JS page
Entry point: "Test a new entry"	Website panel to submit your own entry and watch it go through the pipeline. Start from a real dataset row (pick a class or random), edit any of the 22 fields, then send once or as a burst. A tracker card shows each stage live: sent to Kafka → detected as <class> (confidence, ms) → incident opened / joined / below guard → agent report. Also: POST /api/ingest (JSON) for scripts, "Launch attack" buttons, and a CLI inject script.	New: one shared inject() used by all four

4. Kafka topics and messages

Topic	Key	Message (JSON)
telemetry.raw	satellite id	seq, timestamp, the 22 model features, ground-truth label (for evaluation only)
telemetry.scored	satellite id	everything above + predicted_class, confidence, detector latency (ms)
incidents	incident id	incident id, class, status (open/closed), first/last seen, row count, peak confidence, sample rows
reports	incident id	incident id, SPARTA techniques, mitigation steps, full incident report text, agent runtime

4b. Entry point: testing a new entry

The prototype's demo feature. A visitor builds an entry, sends it into the real Kafka pipeline (no shortcut), and watches how Sentinel treats it, stage by stage.



- **Load sample** fills the form with a real dataset row of the chosen class, so values are realistic. Any field can then be edited to see how the prediction changes.
- **Send once:** the entry is detected and classified instantly, but shows "below incident threshold" (the 3-in-a-row guard), so no AI report is generated. **Send as a burst:** the full chain runs, through to incident and agent report.

- Other entry points share the same inject() function: **Launch attack** buttons (one click), **POST /api/ingest** (JSON, for scripts or a real feed later), and a **CLI script** for testing.
- During demos the replay runs in normal-only mode, so attacks appear only when someone injects them.

5. Today's schedule

Durations are estimates. Each checkpoint is tested live before moving on, and you review the result at three points (marked **REVIEW**), so you can stop or redirect at any time.

#	Step	Est.	Done when
1	Docker Compose: Kafka + Kafka UI; topic creation	45 min	Broker healthy; 4 topics visible in Kafka UI
2	Shared streaming config + Producer (normal-only mode) + inject() + CLI inject script	1 h	Rows visibly arriving in telemetry.raw; a CLI-injected entry appears too
3	Detector service	45 min	Every row in telemetry.scored has a class + confidence; prediction accuracy vs ground truth printed
	REVIEW 1: data flowing end to end through Kafka		
4	Incident manager (grouping + open/close + false-alarm guards)	1 h	One attack burst = exactly one incident; a single stray or low-confidence attack row opens none
5	Agent worker + SQLite storage	1 h	A detected incident automatically gets a full agent report, saved to SQLite
	REVIEW 2: incidents analysed automatically		
6	FastAPI website: backend + WebSocket + page	2.5 h	Live chart, incident feed and reports update in the browser without refresh
6b	Entry point: "Test a new entry" panel, POST /api/ingest, attack buttons	45 min	A submitted entry is tracked live from Kafka to detected class to incident to report
7	One-command start (start script), README section	30 min	Fresh terminal: one command brings the whole system up
8	Remove Hugging Face Space deployment	15 min	deploy-hf-space.yml and HF-only files removed; nothing else broken
9	Full end-to-end test + browser test (Playwright)	45 min	All 5 classes detected and reported; no errors in any service log
	REVIEW 3: final demo, then commit + push under your name		

Total: about 9–10 hours of work. Stretch goal if time remains: **live window features** (see section 8).

6. Files

Path	Status
docker-compose.yml	new
src/streaming/config.py, producer.py, inject.py, detector.py, incident_manager.py, agent_worker.py, storage.py	new
src/web/server.py, src/web/static/index.html (+ app.js, style.css)	new
scripts/start_all (one-command launcher)	new
requirements.txt	add confluent-kafka, fastapi, uvicorn
README.md, docs/ARCHITECTURE.md	new real-time section
.github/workflows/deploy-hf-space.yml and HF-only files	removed (step 8)

Path	Status
src/dashboard/app.py, src/pipeline/*, src/agents/*	unchanged (reused, not edited)

7. How it will run

```
docker compose up -d          # Kafka + Kafka UI
scripts/start_all             # producer, detector, incidents, agents, website
open http://localhost:8000    # Sentinel live website
open http://localhost:8080    # Kafka UI (topics, messages, consumer lag)
```

Requires Docker Desktop to be running. Docker 29.8 and Compose v5.6 are already installed.

8. Scope limits and risks

Item	Impact	Handling
Groq free tier: 8,000 tokens/min	Each agent report takes ~1–2 min; several incidents queue up	Kafka queues them; worker processes one at a time with retries. Detection itself is instant and unaffected.
Window features are precomputed in the CSV	Producer replays rows that already contain window features	Honest v1. Stretch goal: compute the sliding-window features live inside the detector from raw messages.
Class decided by XGBoost only	Wrong prediction = wrong incident and report; unseen attack types map to the nearest known class	Confidence + minimum-burst guards cut false incidents; live accuracy vs ground truth shown on the website; closed-set limit stated on poster.
Docker Desktop not running	Kafka won't start	Start script checks and prints a clear message.
Website is the largest step	Most likely to overrun	Core views first (chart, incidents, reports); styling last.
History	SQLite is local to one machine	Fine for a poster demo; Postgres later if hosted.

9. Poster talking points

- Event-driven streaming architecture on Apache Kafka: decoupled producer, detector, incident and agent services.
- Two-speed design: millisecond ML detection on every message; slower LLM agents only on deduplicated incidents. Groq cost scales with the number of attacks, not the volume of telemetry (one crew run per incident).
- Back-pressure handled by Kafka: the agent tier can lag under API rate limits without losing data or slowing detection.
- Hands-on demo: visitors submit their own entry or launch an attack, and watch Sentinel detect it, open an incident and write the report live.
- Live operator website: telemetry, incidents and AI-generated SPARTA-mapped reports pushed to the browser in real time.
- Measurable: throughput (rows/sec), detection latency, consumer lag, time from attack to finished report.

10. Decision needed

Approve this plan to start at step 1. Each review point is a chance to change direction.